

Website Performance Review

fitnafilter.com

To: FitnaFilter

From: AQL Technology

Date: June 2026

Subject: Website performance review covering launch readiness, asset weight, browser rendering cost, runtime behaviour, and optimization priorities

1. Executive Summary

The local FitnaFilter homepage has a strong first impression. The hero is clear, product-specific, and visually distinctive: a privacy-first browser extension that filters explicit imagery, blocks harmful domains, and redirects users toward better choices. The design language is confident and memorable, and the first viewport communicates the product well on both desktop and mobile.

The main issue is not weak design. It is that the page is carrying a heavy decorative performance load. The homepage currently ships a 2.06 MB hero image, requests a broad Google Fonts set, embeds a large inline SVG logo sprite, animates a full-viewport canvas background, and keeps multiple decorative motion systems alive at once. Much of this can be improved without changing the visible output.

The other important issue is deployment readiness. As checked on 2026-06-05, the HTTPS apex returned Cloudflare 522, while the HTTP apex redirected to `www`, which served a Sedo/parking-style page. That means the live domain is not yet serving the local homepage build reviewed here.

Overall Grade for local homepage build: B

Visually strong and strategically coherent, but performance polish and launch-domain configuration need attention before public release.

Category	Count	Commentary
Launch blocker	1	Live-domain issue preventing the reviewed homepage from being served at <code>fitnafilter.com</code> .
Free Lunch	8	Exact same visible output with lower network, runtime, or rendering cost.
Minor Changes	6	Small visual or timing changes that are likely worth it.
Major Changes	3	Larger design or architecture changes to consider after the conservative pass.

2. Scope and Method

This review is based on the local FitnaFilter website build in the repo's `website/` directory. The method included a local static-server run, Brave remote debugging on `127.0.0.1:9222`, source inspection of `home.html`, `home.css`, and `home.js`, asset-size review, browser resource timing inspection, responsive screenshot capture, live-domain `curl` checks, and parallel read-only review of HTML/network, CSS/rendering, JS/runtime, and UX trade-off areas.

Important context: The live domain is not currently serving the local homepage reviewed in this document. Deployment/DNS readiness should be addressed before treating public traffic, search previews, or production Lighthouse results as representative of the FitnaFilter homepage.

3. Asset and Runtime Snapshot

Asset	Raw Size
<code>website/hero_art.png</code>	2,061,023 bytes
<code>website/home.html</code>	76,468 bytes
<code>website/home.css</code>	61,813 bytes
<code>website/home.js</code>	26,971 bytes

Additional browser observations:

- Inline SVG sprite block: about 30.5 KB raw.
- Google Fonts request: 6 families and a wide weight set; live CSS expanded to about 38.8 KB and 69 `@font-face` rules during the browser run.
- Runtime DOM after initialization: 835 elements.

- Decorative generated elements: 9 hero drift chips, 21 portal chips, 9 playground tiles, and 12 ayah cards after marquee cloning.
- Full-viewport canvas measured 1440 x 900 on the desktop capture and 390 x 844 on the mobile capture.

4. Prioritised Findings

4.1. Launch Blocker

1. The public domain is not serving the reviewed homepage

`https://fitnafilter.com` returned Cloudflare 522. `http://fitnafilter.com` redirected via Namecheap URL forwarding to `http://www.fitnafilter.com/`, and `www.fitnafilter.com` served a parking-style page. Visitors will not see the FitnaFilter homepage, and search engines or social previews may index the wrong experience. *Recommendation:* Point the apex and `www` DNS records to the intended host, configure HTTPS, remove parking/forwarding, and re-test all HTTP/HTTPS apex and `www` variants.

4.2. Free Lunch: Same Output, More Efficient

2. Trim the Google Fonts request

The page requests Sora, Inter, Instrument Serif, JetBrains Mono, Noto Naskh Arabic, and Amiri Quran, with multiple weights. Instrument Serif is defined but not visibly used, and Amiri Quran appears only as a fallback.

Recommendation: Remove unused families and request only the used weights. This keeps the rendered page the same while reducing CSS/font discovery and downloads.

3. Losslessly optimize the hero image

The preloaded CSS hero background is a 2.06 MB PNG. This is the largest local asset by a wide margin.

Recommendation: Run lossless PNG recompression and consider lossless WebP with PNG fallback. This keeps the same visible image while reducing transfer size.

4. Skip canvas edge-glow scanning when the pointer is inactive

The decorative tessellation canvas still enters the edge-highlight loop even when the pointer sentinel is offscreen.

Recommendation: Short-circuit the pointer glow loop unless the pointer is active and inside the viewport. Keep the base path and dots unchanged.

5. Pause canvas animation while the document is hidden

The canvas uses continuous `requestAnimationFrame` while motion is enabled.

Recommendation: Use `visibilitychange` to stop rAF work in background tabs and resume on return. Visible output is unchanged.

6. Convert layout-affecting animations to transform-based animations

Portal chips animate `left`, the scroll progress bar updates `width`, and nav underline hover transitions `width`.

Recommendation: Use `transform: translate3d(...)` for chip travel and `transform: scaleX(...)` for the progress bar and underline. The visual effect can match the current page while avoiding per-frame layout work.

7. Tighten scroll and drag hot paths

Sticky nav and scroll progress are separate scroll handlers. The before/after slider reads layout on every pointer move and listens globally even when not dragging.

Recommendation: Combine scroll work in one rAF-scheduled handler, cache scroll maximum on resize, cache the slider rect per drag, and attach `pointermove` only during active drag.

8. Reduce repeated DOM parsing and querying

Counters parse dataset values repeatedly, playground sorting and DOM querying are repeated in a small interactive area, and tile segment generation can repeat on resize.

Recommendation: Pre-parse counter metadata, precompute ranked playground tiles, cache swatch/toggle node lists, and cache `tileSegments()` by tile size.

9. Remove dead or no-op CSS

Selectors such as `.feature-grid`, `.card--wide`, and empty playground-level rules do not affect the current page.

Recommendation: Remove dead selectors and merge duplicated breakpoint blocks for a small parse/rule-match win.

4.3. Minor Changes: Small Output Changes, Likely Worth It

10. Use lossy WebP or AVIF for the hero

A high-quality lossy hero could reduce the image from megabytes to a small fraction of the current size. Pixels change slightly, so this belongs below the exact-output optimizations.

11. Reduce decorative blended layers on low-power or mobile contexts

The page uses a full-screen blended canvas, a fixed pointer glow, and a fixed backdrop-filter nav. These are visually appealing but expensive to composite.

Recommendation: Disable or simplify the cursor glow and blended canvas on mobile/low-power modes, and consider replacing the nav blur with a solid translucent background.

12. Pause offscreen decorative motion

Hero drift chips, portal chips, marquee rows, and ayah rail animation add constant visual work.

Recommendation: Start these systems only when their section is near viewport, or pause them when far offscreen.

13. Remove or simplify low-value pointer effects

Hero parallax, card spotlight tracking, and pointer glow are subtle but keep pointer work active.

Recommendation: Keep the hero image and core brand signal, but remove or gate the pointer flourishes.

14. Choose either hero drift chips or the marquee

Both communicate a similar blocked-domain flow idea. Keeping both costs extra animation and visual noise.

Recommendation: Keep the stronger brand signal and remove the weaker duplicate if the page needs to feel calmer.

15. Tighten reduced-motion handling

The current reduced-motion CSS compresses animation durations, but many decorative layers still exist.

Recommendation: Hide or freeze cursor glow, hero drift, portal chips, the content canvas, and marquee-like rails for reduced-motion users.

4.4. Major Changes

16. Replace the animated canvas background

This is the largest CPU and battery improvement candidate, but it removes the interactive tessellation atmosphere, one of the page's more distinctive visual traits.

17. Convert the CSS hero background to a responsive image system

A responsive `picture/img` implementation would improve source selection and browser priority handling. It is a structural rewrite with stacking, overlay, and parallax QA risk.

18. Merge the playground and before/after demo

This would remove a full visual block and some interaction code, but it changes content strategy. The page currently benefits from having both an interactive control demo and a simple before/after metaphor.

5. Recommended Implementation Order

#	Action	Type
1	Fix live-domain routing and HTTPS so the right site is served.	Launch
2	Trim fonts and losslessly optimize hero assets.	Free Lunch
3	Add canvas hidden-tab pause and inactive-pointer short-circuit.	Free Lunch
4	Convert <code>left</code> and <code>width</code> animations to transforms.	Free Lunch
5	Consolidate scroll handlers and slider pointer handling.	Free Lunch
6	Remove dead CSS and strip unused SVG export metadata.	Free Lunch
7	Re-test desktop and mobile screenshots for no visible regressions.	QA
8	Decide whether to adopt lossy hero formats and reduced decorative motion.	Minor

6. Commercial and Product Implications

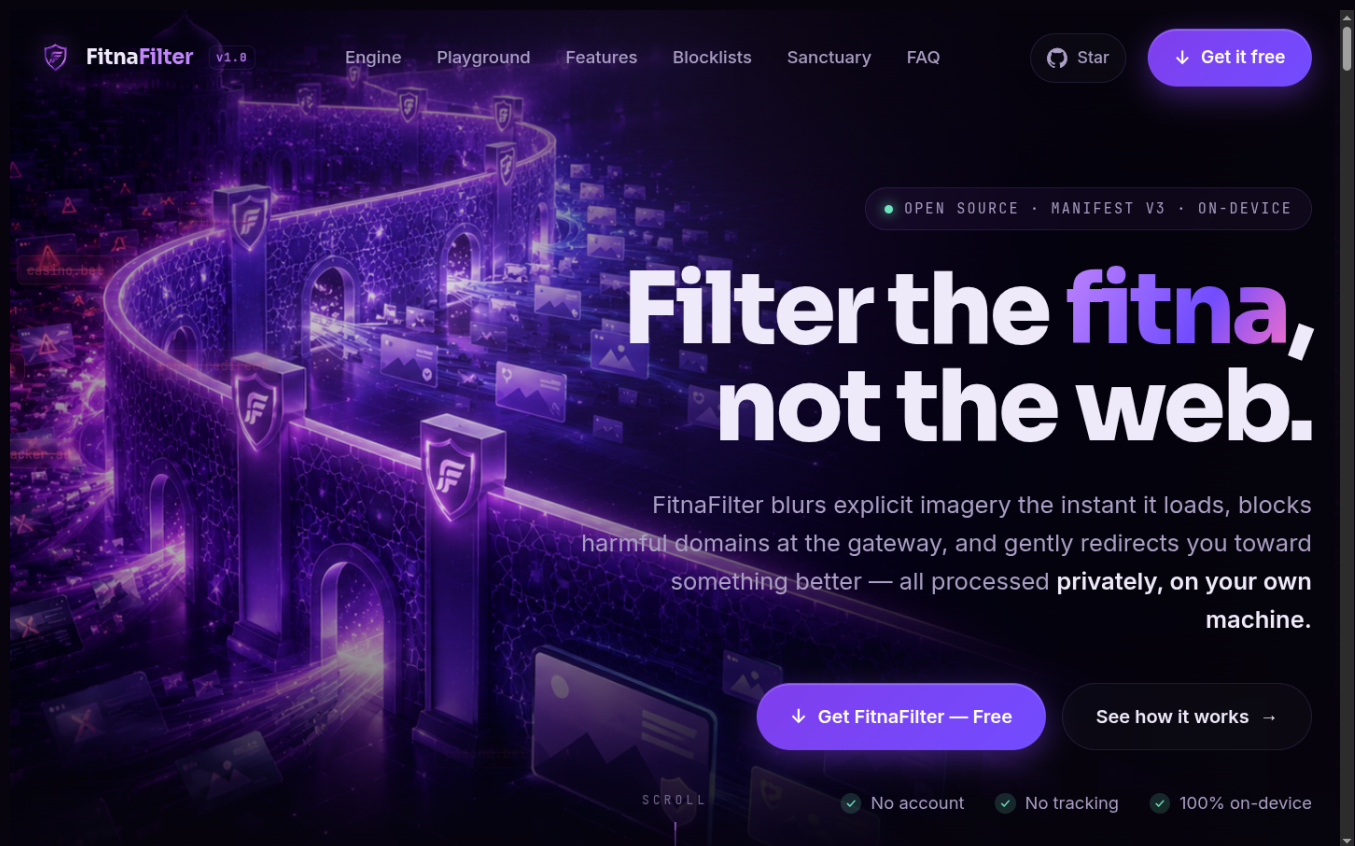
FitnaFilter's homepage is already stronger than a generic extension landing page. It has a distinctive hero, clear value proposition, strong privacy framing, and product-specific interaction demos. The optimization work should therefore preserve the brand signal rather than flattening the site.

The best first pass is conservative: keep the page looking the same, but cut obvious network and runtime waste. After that, decide how much decorative motion is worth keeping on mobile and lower-power contexts.

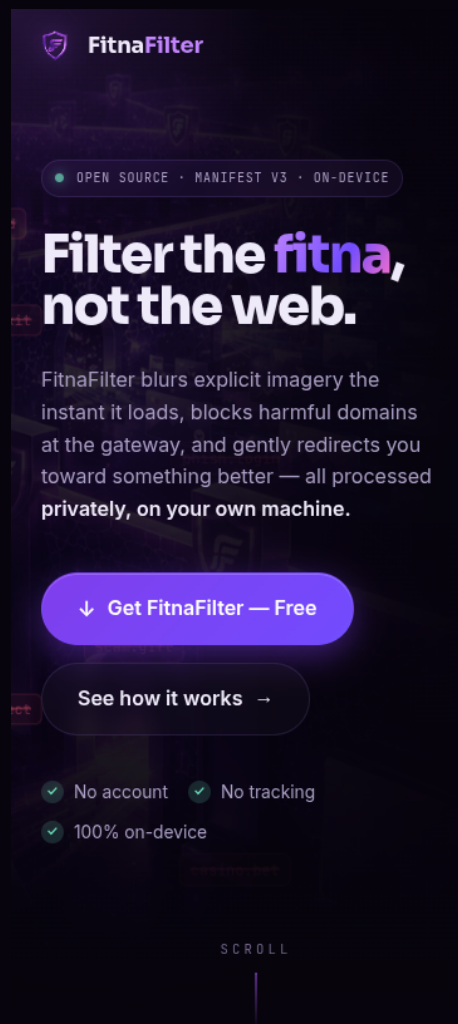
7. Evidence Register

Ref	File	Description
E1	evidence/01-local-home-desktop-hero.png	Desktop hero screenshot of the inspected local homepage.
E2	evidence/02-local-home-mobile-hero.png	Mobile hero screenshot at 390px width.
E3	evidence/03-local-home-desktop-page-top.png	Full-width top-page capture used as visual context for the local build.
E4	evidence/capture-summary.json	Browser-side DOM and decorative-element counts captured through Brave/CDP.

8. Screenshot Appendix



E1. Desktop homepage hero. Strong brand signal and clear value proposition, but backed by a large hero image and decorative rendering stack.



E2. Mobile homepage hero. The local build remains visually coherent on mobile, while still carrying the full decorative system.



E3. Top-page context. The inspected build is a single-page marketing site with several animated and interactive sections below the hero.